

# Algorithmique et programmation

*Lance-toi dans la programmation ! Dans cette toute première activité, tu vas apprendre à manipuler des nombres, des variables et tu vas coder tes premières boucles avec Python.*

## Cours 1 (Nombres avec Python).

Vérifie dans la console que Python fonctionne correctement, en tapant les commandes suivantes dans une console Python :

```
>>> 2+2
>>> "Bonjour le monde !"
```

Voici quelques instructions.

- **Addition.**  $5+7$ .
- **Multipliation.**  $6*7$  ; avec des parenthèses  $3*(12+5)$  ; avec des nombres à virgule  $3*1.5$ .
- **Puissance.**  $3**2$  pour  $3^2 = 9$  ; puissance négative  $10**-3$  pour  $10^{-3} = 0.001$ .
- **Division réelle.**  $14/4$  vaut  $3.5$  ;  $1/3$  vaut  $0.3333333333333333$ .
- **Division entière et modulo.**
  - $14//4$  vaut  $3$  : c'est le quotient de la division euclidienne de  $14$  par  $4$ , note bien la double barre ;
  - $14\%4$  vaut  $2$  : c'est le reste de la division euclidienne de  $14$  par  $4$ , on dit aussi «  $14$  modulo  $4$  ».

*Remarque.* Dans tout ce cours, on écrira les « nombres à virgule » sous la forme  $3.5$  (et pas  $3,5$ ). Le séparateur décimal est donc le point. En informatique les nombres à virgule sont appelés « nombres flottants ».

## Activité 1 (Premiers pas).

*Objectifs : faire tes premiers calculs avec Python.*

1. Combien y a-t-il de secondes en un siècle ? (Ne tiens pas compte des années bissextiles.)
2. Jusqu'où faut-il compléter les pointillés pour obtenir un nombre plus grand que  $150$  ?

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + \dots$$

3. Quel est le dernier chiffre de  $2^{64}$  ?

## Cours 2 (Travailler avec un éditeur).

À partir de maintenant, il est préférable que tu travailles dans un éditeur de texte dédié à Python. Tu dois alors explicitement demander d'afficher le résultat :

```
print(2+2)
print("Bonjour le monde !")
```

**Cours 3 (Variables).**

**Variable.** Une *variable* est un nom associé à un emplacement de la mémoire. C'est comme une boîte que l'on identifie par une étiquette. La commande « `a = 3` » signifie que j'ai une variable « `a` » associée à la valeur 3.

Voici un premier exemple :

```
a = 3 # Une variable
b = 5 # Une autre variable

print("La somme vaut",a+b) # Affiche la somme
print("Le produit vaut",a*b) # Affiche le produit

c = b**a # Nouvelle variable...
print(c) # ... qui s'affiche
```

**Commentaires.** Tout texte qui suit le caractère dièse « `#` » n'est pas exécuté par Python mais sert à expliquer le programme. C'est une bonne habitude de commenter abondamment ton code.

**Noms.** Il est très important de donner un nom clair et précis aux variables. Par exemple, avec les noms bien choisis tu devrais savoir ce que calcule le code suivant :

```
base = 8
hauteur = 3
aire = base * hauteur / 2
print(aire)
print(Aire) # !! Erreur !!
```

Attention! Python distingue les majuscules des minuscules. Donc `mavariabLe`, `MavariabLe` et `MAVARIABLE` sont des variables différentes.

**Réaffectation.** Imaginons que tu veuilles tenir tes comptes journaliers. Tu pars d'une somme  $S_0 = 1000$ , le lendemain tu gagnes 100, donc maintenant  $S_1 = S_0 + 100$ ; le jour d'après tu rajoutes 200, donc  $S_2 = S_1 + 200$ ; puis tu perds 50, donc au troisième jour  $S_3 = S_2 - 50$ . Avec Python tu peux n'utiliser qu'une seule variable `S` pour toutes ces opérations.

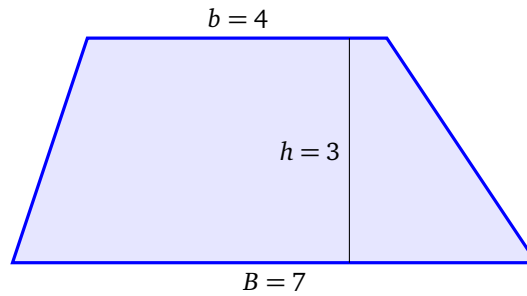
```
S = 1000
S = S + 100
S = S + 200
S = S - 50
print(S)
```

Il faut comprendre l'instruction « `S = S + 100` » comme ceci : « je prends le contenu de la boîte `S`, je rajoute 100, je remets tout dans la même boîte ».

**Activité 2 (Variables).**

*Objectifs : utiliser des variables !*

1. (a) Définis des variables, puis calcule l'aire d'un trapèze. Ton programme doit afficher "L'aire vaut ..." en utilisant `print("L'aire vaut",aire)`.



- (b) Définis des variables pour calculer le volume d'une boîte (un parallélépipède rectangle) dont les dimensions sont 10, 8, 3.
- (c) Définis une variable PI qui vaut 3.14. Définis un rayon  $R = 10$ . Écris la formule de l'aire du disque de rayon  $R$ .
2. Remets les lignes dans l'ordre de sorte qu'à la fin  $x$  ait la valeur 46.

- (1)  $y = y - 1$
- (2)  $y = 2 * x$
- (3)  $x = x + 3 * y$
- (4)  $x = 7$

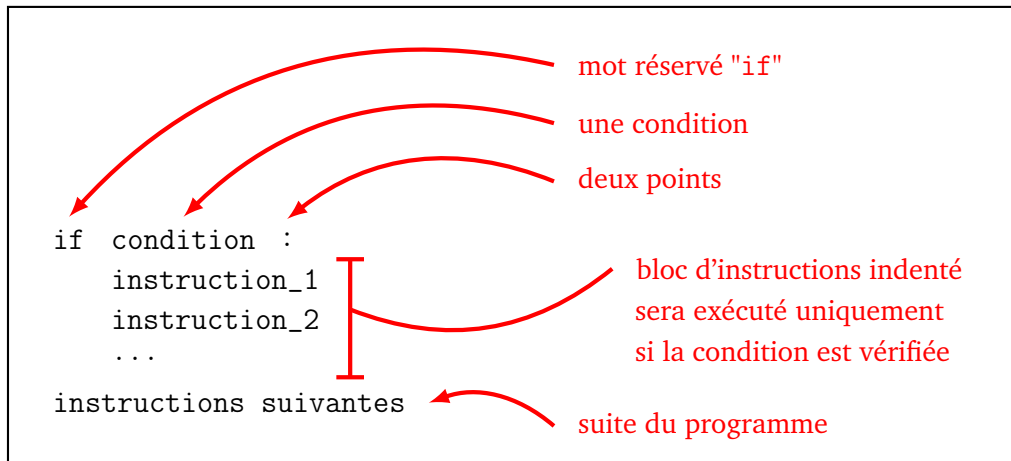
3. Tu places la somme de 1000 euros sur un compte d'épargne. Chaque année les intérêts sur l'argent placé rapportent 10% (le capital est donc multiplié par 1.10). Écris le code qui permet de calculer le capital pour les trois premières années.
4. Je définis deux variables par  $a = 9$  et  $b = 11$ . Je souhaite échanger le contenu de  $a$  et  $b$ . Quelles instructions conviennent de sorte qu'à la fin  $a$  vaut 11 et  $b$  vaut 9?

|         |         |         |         |
|---------|---------|---------|---------|
| $a = b$ | $c = b$ | $c = a$ | $c = a$ |
| $b = a$ | $a = b$ | $a = b$ | $a = c$ |
|         | $b = c$ | $b = c$ | $c = b$ |
|         |         |         | $b = c$ |

*L'ordinateur peut réagir en fonction d'une situation. Si une condition est remplie il agit d'une certaine façon, sinon il fait autre chose.*

#### **Cours 4** (Si ... alors ...).

Voici comment utiliser le test « if » avec Python :



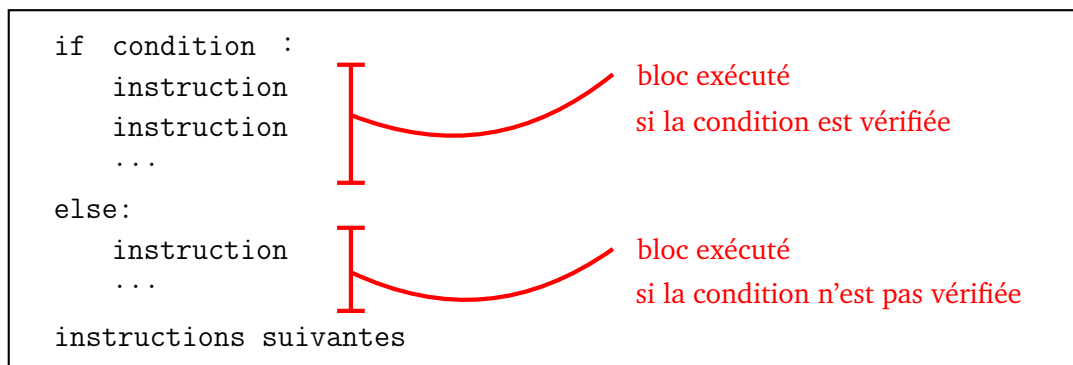
Voici un exemple, qui avertit un conducteur si une variable vitesse est trop grande.

```

if vitesse > 110:
    print("Attention, tu roules trop vite.")

```

On peut aussi exécuter des instructions si la condition n'est pas remplie à l'aide du mot « else ».



Encore une fois c'est l'indentation qui délimite les différents blocs d'instructions. Voici un exemple qui affiche le signe d'un nombre  $x$ .

```

if x >= 0:
    print("Le nombre est positif.")
else:
    print("Le nombre est négatif.")

```

### Cours 5 (Le module « random »).

Le module `random` génère des nombres comme s'ils étaient tirés au hasard.

- Voici la commande à placer au début du programme pour appeler ce module :

```
from random import *
```

- La commande `randint(a,b)` renvoie un entier au hasard compris entre  $a$  et  $b$ .

Par exemple après l'instruction `n = randint(1,6)`,  $n$  est un entier tiré au hasard avec  $1 \leq n \leq 6$ . Si on recommence l'instruction `n = randint(1,6)`,  $n$  prend une nouvelle valeur. C'est comme si on effectuait le lancer d'un dé à 6 faces.

- La commande `random()`, sans argument, renvoie un nombre flottant (c'est-à-dire un nombre à virgule) compris entre 0 et 1. Par exemple, après l'instruction `x = random()`, alors `x` est un nombre flottant avec  $0 \leq x < 1$ .

### Activité 3 (Quiz multiplications).

*Objectifs : programmer un petit test sur les tables de multiplication.*

- Définis une variable `a`, à laquelle tu affectes une valeur au hasard entre 1 et 12.
- Même chose pour une variable `b`.
- Affiche à l'écran la question : « Combien vaut le produit  $a \times b$  ? » (Remplace `a` et `b` par leur valeur !)
- Récupère la réponse de l'utilisateur et transforme-la en un entier.
- Si la réponse est correcte affiche « Bravo ! », sinon affiche « Perdu ! La bonne réponse était... ».

**Test d'égalité.** Pour tester si deux nombres `x` et `y` sont égaux, l'instruction est :

```
if x == y:
```

Le test d'égalité s'écrit bien avec le double signe égal «`==`». Par exemple «`x == 3`» renvoie «vrai» si `x` vaut 3 et «faux» sinon.

Attention ! La commande «`x = 3`» n'a rien à voir, cette instruction stocke 3 dans la variable `x`.

### Cours 6 (Utiliser des fonctions).

- **Utiliser des fonctions de Python.**

Tu connais déjà la fonction `print()` qui affiche une chaîne de caractères (ou des nombres). Elle s'utilise ainsi `print("Coucou")` ou bien à travers une valeur :

```
chaine = "Bonjour"
print(chaine)
```

Il existe plein d'autres fonctions. Par exemple la fonction `abs()` calcule la valeur absolue : `abs(-3)` renvoie 3, `abs(5)` renvoie 5.

- **Le module `math`.**

Toutes les fonctions ne sont pas directement accessibles. Elles sont souvent regroupées dans des **modules**. Par exemple le module `math` contient les fonctions mathématiques. Tu y trouves par exemple la fonction racine carrée `sqrt()` (*square root*). Voici comment l'utiliser :

```
from math import *

x = sqrt(2)
print(x)
print(x**2)
```

La première ligne importe toutes les fonctions du module `math`, la seconde calcule  $x = \sqrt{2}$  (en valeur approchée) et ensuite on affiche `x` et `x2`.

- **Sinus et cosinus.**

Le module `math` contient les fonctions trigonométriques sinus et cosinus et même la constante `pi` qui est une valeur approchée de  $\pi$ . Attention, les angles sont exprimés en radians.

Voici le calcul de  $\sin(\frac{\pi}{2})$ .

```
angle = pi/2
print(angle)
print(sin(angle))
```

• **Décimal vers entier.**

Dans le module `math` il y a aussi des fonctions pour arrondir un nombre décimal :

- `round()` arrondit à l'entier le plus proche : `round(5.6)` renvoie 6, `round(1.5)` renvoie 2.
- `floor()` renvoie l'entier inférieur ou égal : `floor(5.6)` renvoie 5.
- `ceil()` renvoie l'entier supérieur ou égal : `ceil(5.6)` renvoie 6.

**Activité 4** (Utiliser des fonctions).

*Objectifs : utiliser des fonctions de Python et du module `math`.*

1. Écrire une fonction *absolue* qui donne la valeur absolue d'un nombre. vérifier que pour tout nombre  $a$ ,  $abs(a) = absolue(a)$ .
2. Trouve par tâtonnement un nombre réel  $x$  qui vérifie toutes les conditions suivantes (plusieurs solutions sont possibles) :
  - `abs(x**2 - 15)` est inférieur à 0.5
  - `round(2*x)` renvoie 8
  - `floor(3*x)` renvoie 11
  - `ceil(4*x)` renvoie 16

*Indication.* `abs()` désigne la fonction valeur absolue.

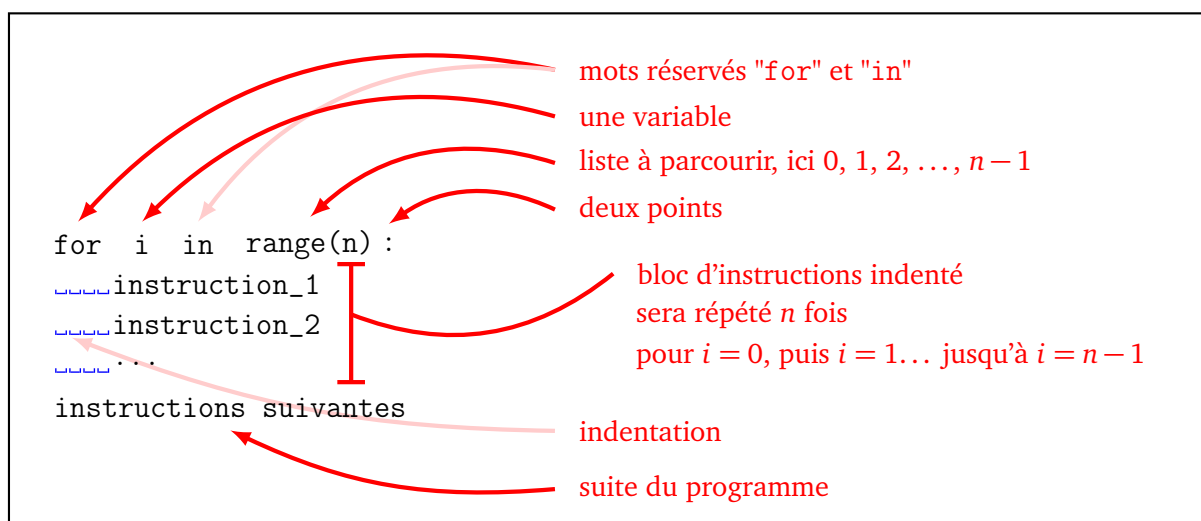
3. Tu connais la formule de trigonométrie

$$\cos^2 \theta + \sin^2 \theta = 1.$$

Vérifie que pour  $\theta = \frac{\pi}{7}$  (ou d'autres valeurs) cette formule est numériquement vraie. (Ce n'est pas une preuve de la formule, car Python ne fait que des calculs approchés du sinus et du cosinus).

**Cours 7** (Boucle « pour »).

*La boucle « pour » est la façon la plus simple de répéter des instructions.*



Note bien que ce qui délimite le bloc d'instructions à répéter, c'est **l'indentation**, c'est-à-dire les espaces

placées en début de ligne qui décalent les lignes vers la droite. Toutes les lignes d'un bloc doivent avoir exactement la même indentation. Dans ce livre, nous choisissons une indentation de 4 espaces.

N'oublie pas les deux points en fin de la ligne de la déclaration du `for` !

- **Exemple de boucle « pour ».**

Voici une boucle qui affiche les premiers carrés.

```
for i in range(10):
    print(i*i)
```

La seconde ligne est décalée et constitue le bloc à répéter. La variable `i` prend la valeur 0 et l'instruction affiche  $0^2$  ; puis `i` prend la valeur 1, et l'instruction affiche  $1^2$  ; puis  $2^2$ ,  $3^2$ ...

Au final ce programme affiche :

0, 1, 4, 9, 16, 25, 36, 49, 64, 81.

Attention : la dernière valeur prise par `i` est bien 9 (et pas 10).

- **Parcourir une liste quelconque.**

La boucle « pour » permet de parcourir n'importe quelle liste. Voici une boucle qui affiche le cube des premiers nombres premiers.

```
for p in [2,3,5,7,11,13]:
    print(p**3)
```

- **Sommes des entiers.**

Voici un programme qui calcule

$0 + 1 + 2 + 3 + \dots + 18 + 19$ .

```
somme = 0
for i in range(20):
    somme = somme + i
print(somme)
```

Comprends bien ce code : une variable `somme` est initialisée à 0. On va tour à tour lui ajouter 0, puis 1, puis 2... On peut mieux comprendre cette boucle en complétant un tableau :

Initialisation : `somme = 0`

| i   | somme |
|-----|-------|
| 0   | 0     |
| 1   | 1     |
| 2   | 3     |
| 3   | 6     |
| 4   | 10    |
| ... | ...   |
| 18  | 171   |
| 19  | 190   |

Affichage : 190

- `range()`.
  - Avec `range(n)` on parcourt les entiers de 0 à  $n - 1$ . Par exemple `range(10)` correspond à la liste `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`.  
Attention ! la liste s'arrête bien à  $n - 1$  et pas à  $n$ . Ce qu'il faut retenir c'est que la liste contient bien  $n$  éléments (car elle commence à 0).
  - Si tu veux afficher la liste des éléments parcourus, il faut utiliser la commande :  
`list(range(10))`
  - Avec `range(a,b)` on parcourt les éléments de  $a$  à  $b - 1$ . Par exemple `range(10,20)` correspond à la liste `[10, 11, 12, 13, 14, 15, 16, 17, 18, 19]`.
  - Avec `range(a,b,pas)` on parcourt les éléments  $a, a + \text{pas}, a + 2\text{pas} \dots$ . Par exemple `range(10,20,2)` correspond à la liste `[10, 12, 14, 16, 18]`.

### Activité 5 (Boucle « pour »).

*Objectifs : construire des boucles simples.*

1. (a) Affiche les cubes des entiers de 0 à 100.  
(b) Affiche les puissances quatrièmes des entiers de 10 à 20.  
(c) Affiche les racines carrées des entiers 0, 5, 10, 15, ... jusqu'à 100.
2. Affiche les puissances de 2, de  $2^1$  à  $2^{10}$ , et apprends par cœur les résultats !
3. Recherche de façon expérimentale une valeur approchée du minimum de la fonction

$$f(x) = x^3 - x^2 - \frac{1}{4}x + 1$$

sur l'intervalle  $[0, 1]$ .

*Indications.*

- Construis une boucle dans laquelle une variable  $i$  balaye les entiers de 0 à 100.
  - Définis  $x = \frac{i}{100}$ . Ainsi  $x = 0.00$ , puis  $x = 0.01$ ,  $x = 0.02 \dots$
  - Calcule  $y = x^3 - x^2 - \frac{1}{4}x + 1$ .
  - Affiche les valeurs à l'aide de `print("x =", x, "y =", y)`.
  - Cherche à la main pour quelle valeur de  $x$  on obtient un  $y$  le plus petit possible.
  - N'hésite pas à modifier ton programme pour augmenter la précision.
4. Cherche une valeur approchée que doit avoir le rayon  $R$  d'une boule afin que son volume soit 100.

*Indications.*

- Utilise une méthode de balayage comme à la question précédente.
- La formule du volume d'une boule est  $V = \frac{4}{3}\pi R^3$ .
- Affiche les valeurs à l'aide de `print("R =", R, "V =", V)`.
- Pour  $\pi$  tu peux prendre la valeur approchée 3.14 ou bien la valeur approchée `pi` du module `math`.

### Activité 6 (Boucle « pour » (suite)).

*Objectifs : construire des boucles plus compliquées.*

1. Définis une variable  $n$  (par exemple  $n = 20$ ). Calcule la somme

$$1^2 + 2^2 + 3^2 + \dots + i^2 + \dots + n^2.$$

2. Calcule le produit :

$$1 \times 3 \times 5 \times \dots \times 19.$$

*Indications.* Commence par définir une variable `produit` initialisée à la valeur 1. Utilise `range(a,b,2)` pour obtenir un entier sur deux.

3. Affiche les tables de multiplication entre 1 et 10. Voici un exemple de ligne à afficher :

$$7 \times 9 = 63$$

Utilise une commande d'affichage du style : `print(a,"x",b,"=",a*b)`.

## Cours 8 (Booléens).

- Un **booléen** est une donnée qui vaut soit la valeur « vrai », soit la valeur « faux ». En Python les valeurs sont `True` et `False` (avec une majuscule).
- On obtient un booléen par exemple comme résultat de la comparaison de deux nombres. Par exemple `7 < 4` vaut `False` (car 7 n'est pas plus petit que 4). Vérifie que `print(7 < 4)` affiche `False`.

Voici les principales comparaisons :

- **Test d'égalité** : `a == b`
- **Test inférieur strict** : `a < b`
- **Test inférieur large** : `a <= b`
- **Test supérieur** : `a > b` ou `a >= b`
- **Test non égalité** : `a != b`

Par exemple `6*7 == 42` vaut `True`.

•

**ATTENTION !** L'erreur classique est de confondre « `a = b` » et « `a == b` ».

— **Affectation.** `a = b` met le contenu de la variable `b` dans la variable `a`.

— **Test d'égalité.** `a == b` teste si les contenus de `a` et de `b` sont égaux et vaut `True` ou `False`.

- On peut comparer autre chose que des nombres. Par exemple « `car == "A"` » teste si la variable `car` vaut "A"; « `il_pleut == True` » teste si la variable `il_pleut` est vraie...
- Les booléens sont utiles dans le test « si ... alors ... » et dans les boucles « tant que ... alors ... ».
- **Opérations entre les booléens.** Si `P` et `Q` sont deux booléens, on peut définir de nouveaux booléens.
  - **Et logique.** « `P and Q` » est vrai si et seulement si `P` et `Q` sont vrais.
  - **Ou logique.** « `P or Q` » est vrai si et seulement si `P` ou `Q` est vrai.
  - **Négation.** « `not P` » est vrai si et seulement si `P` est faux.

Par exemple « `(2+2 == 2*2) and (5 < 3)` » renvoie `False`, car même si on a bien `2 + 2 = 2 × 2`, l'autre condition n'est pas remplie car `5 < 3` est faux.

## Activité 7 (Chiffres d'un nombre).

*Objectifs : trouver des nombres dont les chiffres vérifient certaines propriétés.*

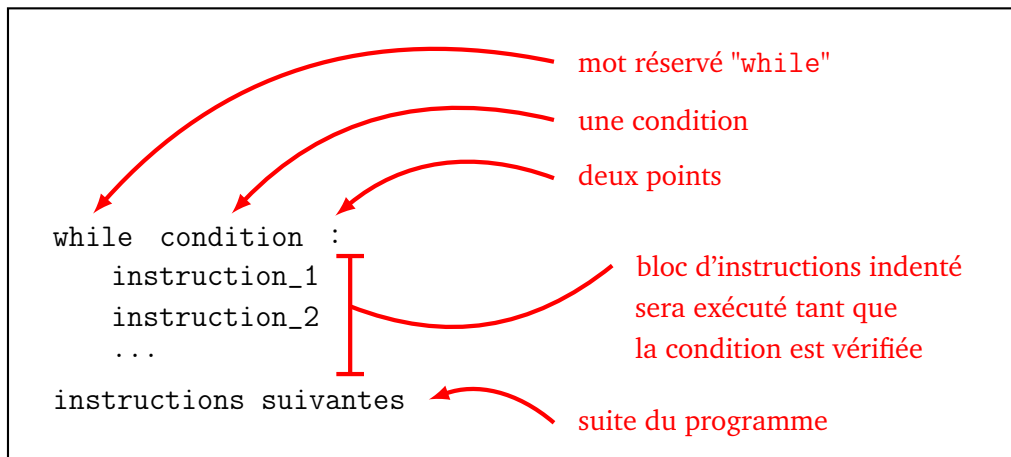
1. Le programme suivant affiche tous les entiers de 0 à 99. Comprends ce programme. Que représentent les variables `u` et `d` ?

```
for d in range(10):
    for u in range(10):
        n = 10*d + u
        print(n)
```

2. Trouve tous les entiers compris entre 0 et 999 qui vérifient toutes les propriétés suivantes :
  - l'entier se termine par 3,
  - la somme des chiffres est supérieure ou égale à 15,
  - le chiffre des dizaines est pair.
3. Modifie ton programme précédent pour compter et afficher le nombre d'entiers vérifiant les propriétés.

### Cours 9 (Boucle « tant que »).

La boucle « tant que » exécute des instructions tant qu'une condition est vraie. Dès que la condition devient fausse, elle passe aux instructions suivantes.



#### Exemple.

Voici un programme qui affiche le compte à rebours 10, 9, 8, ..., 3, 2, 1, 0. Tant que la condition  $n \geq 0$  est vraie, on diminue  $n$  de 1. La dernière valeur affichée est  $n = 0$ , car ensuite  $n = -1$  et la condition «  $n \geq 0$  » devient fausse donc la boucle s'arrête.

```
n = 10
while n >= 0:
    print(n)
    n = n - 1
```

On résume ceci sous la forme d'un tableau :

Entrée :  $n = 10$

| $n$ | « $n \geq 0$ » ? | nouvelle valeur de $n$ |
|-----|------------------|------------------------|
| 10  | oui              | 9                      |
| 9   | oui              | 8                      |
| ... | ...              | ...                    |
| 1   | oui              | 0                      |
| 0   | oui              | -1                     |
| -1  | non              |                        |

Affichage : 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0

**Exemple.**

Ce bout de code cherche la première puissance de 2 plus grande qu'un entier  $n$  donné. La boucle fait prendre à  $p$  les valeurs 2, 4, 8, 16, ... Elle s'arrête dès que la puissance de 2 est supérieure ou égale à  $n$ , donc ici ce programme affiche 128.

```
n = 100
p = 1
while p < n:
    p = 2 * p
print(p)
```

Entrées :  $n = 100, p = 1$

| $p$ | « $p < n$ » ? | nouvelle valeur de $p$ |
|-----|---------------|------------------------|
| 1   | oui           | 2                      |
| 2   | oui           | 4                      |
| 4   | oui           | 8                      |
| 8   | oui           | 16                     |
| 16  | oui           | 32                     |
| 32  | oui           | 64                     |
| 64  | oui           | 128                    |
| 128 | non           |                        |

Affichage : 128

**Exemple.**

Pour cette dernière boucle on a déjà programmé une fonction `est_pair(n)` qui renvoie True si l'entier  $n$  est pair et False sinon. La boucle fait donc ceci : tant que l'entier  $n$  est pair,  $n$  devient  $n/2$ . Cela revient à supprimer tous les facteurs 2 de l'entier  $n$ . Comme ici  $n = 56 = 2 \times 2 \times 2 \times 7$ , ce programme affiche 7.

```
n = 56
while est_pair(n) == True:
    n = n // 2
print(n)
```

Entrée :  $n = 56$

| $n$ | « $n$ est pair » ? | nouvelle valeur de $n$ |
|-----|--------------------|------------------------|
| 56  | oui                | 28                     |
| 28  | oui                | 14                     |
| 14  | oui                | 7                      |
| 7   | non                |                        |

Affichage : 7

Pour ce dernier exemple il est beaucoup plus naturel de démarrer la boucle par

```
while est_pair(n):
```

En effet `est_pair(n)` est déjà une valeur « vrai » ou faux ». On se rapproche d'une phrase « tant que  $n$  est pair... »

**Opération « += ».** Pour incrémenter un nombre tu peux utiliser ces deux méthodes :

```
nb = nb + 1    ou    nb += 1
```

La seconde écriture est plus courte mais rend le programme moins lisible.

**Activité 8** (Nombres premiers).

*Objectifs : tester si un entier est (ou pas) un nombre premier.*

**1. Plus petit diviseur.**

Programme une fonction `plus_petit_diviseur(n)` qui renvoie, le plus petit diviseur  $d \geq 2$  de l'entier  $n \geq 2$ .

Par exemple `plus_petit_diviseur(91)` renvoie 7, car  $91 = 7 \times 13$ .

**Méthode.**

- On rappelle que  $d$  divise  $n$  si et seulement si  $n \% d$  vaut 0.
- La mauvaise idée est d'utiliser une boucle « pour  $d$  variant de 2 à  $n$  ». En effet, si par exemple on sait que 7 est diviseur de 91 cela ne sert à rien de tester si 8, 9, 10... sont aussi des diviseurs car on a déjà trouvé le plus petit.
- La bonne idée est d'utiliser une boucle « tant que » ! Le principe est : « tant que je n'ai pas obtenu mon diviseur, je continue de chercher ». (Et donc, dès que je l'ai trouvé, j'arrête de chercher.)
- En pratique voici les grandes lignes :
  - Commence avec  $d = 2$ .
  - Tant que  $d$  ne divise pas  $n$  alors, passe au candidat suivant ( $d$  devient  $d + 1$ ).
  - À la fin  $d$  est le plus petit diviseur de  $n$  (dans le pire des cas  $d = n$ ).

**2. Nombres premiers (1).**

Modifie légèrement ta fonction `plus_petit_diviseur(n)` pour écrire une première fonction `est_premier_1(n)` qui renvoie « vrai » (True) si  $n$  est un nombre premier et « faux » (False) sinon.

Par exemple `est_premier_1(13)` renvoie True, `est_premier_1(14)` renvoie False.

**3. Nombres premiers (2).**

Améliore ta fonction en une fonction `est_premier_2(n)` qui ne teste pas tous les diviseurs  $d$  jusqu'à  $n$ , mais seulement jusqu'à  $\sqrt{n}$ .

*Explications.*

- Par exemple pour tester si 101 est un nombre premier, il suffit de voir s'il admet des diviseurs parmi 2, 3, ..., 10. Le gain est appréciable !
- Cette amélioration est due à la proposition suivante : si un entier n'est pas premier alors il admet un diviseurs  $d$  qui vérifie  $2 \leq d \leq \sqrt{n}$ .
- Au lieu de tester si  $d \leq \sqrt{n}$ , il est plus facile de tester  $d^2 \leq n$  !

**Cours 10** (Temps de calcul).

Il existe deux façons de faire tourner plus rapidement des programmes : une bonne et une mauvaise. La mauvaise, c'est d'acheter un ordinateur plus puissant. La bonne, c'est de trouver un algorithme plus efficace !

Avec Python, c'est facile de mesurer le temps d'exécution d'une fonction afin de le comparer avec le temps d'exécution d'une autre. Il suffit d'utiliser le module `timeit`.

Voici un exemple : on mesure le temps de calcul de deux fonctions qui ont le même but, tester si un entier  $n$  est divisible par 7.

```
# Première fonction (pas très maligne)
def ma_fonction_1(n):
```

```
    divis = False
    for k in range(n):
        if k*7 == n:
            divis = True
    return divis

# Seconde fonction (plus rapide)
def ma_fonction_2(n):
    if n % 7 == 0:
        return True
    else:
        return False

import timeit

print(timeit.timeit("ma_fonction_1(1000)",
    setup="from __main__ import ma_fonction_1",
    number=100000))
print(timeit.timeit("ma_fonction_2(1000)",
    setup="from __main__ import ma_fonction_2",
    number=100000))
```

### Résultats.

Le résultat dépend de l'ordinateur, mais permet la comparaison des temps d'exécution des deux fonctions.

- La mesure pour la première fonction (appelée 100 000 fois) renvoie 5 secondes. L'algorithme n'est pas très malin. On teste si  $7 \times 1 = n$ , puis on teste  $7 \times 2 = n$ ,  $7 \times 3 = n \dots$
- La mesure pour la seconde fonction renvoie 0.01 seconde ! On teste si le reste de  $n$  divisé par 7 est 0. La seconde méthode est donc 500 fois plus rapide que la première.

### Explications.

- On appelle le module `timeit`.
- La fonction `timeit.timeit()` renvoie le temps d'exécution en seconde. Elle prend comme paramètres :
  - une chaîne pour l'appel de la fonction à tester (ici est-ce que 1000 est divisible par 7),
  - un argument `setup="..."` qui indique où trouver cette fonction,
  - le nombre de fois qu'il faut répéter l'appel à la fonction (ici `number=100000`).
- Il faut que le nombre de répétitions soit assez grand pour éviter les incertitudes.